FleetSync

Complete Java Developer Field Guide

From Zero to Junior Developer in 60 Days

Stack: Spring Boot 3 MySQL JWT REST APIs

Target: Junior Java Developer / Internship

Timeline: 60 Days – 12 Weeks – MVP Ready

Markets: Pakistan UAE Remote

What You Will Build

A production-grade Fleet Management REST API with 3 user roles, JWT authentication, full CRUD, trip lifecycle state machine, and a clean GitHub portfolio for interviews.

Built for Sheikh – 2024–2025 – Version 2.0 (Overleaf Compatible)

Contents

1 How to Use This Guide **4**
- 1.1 The Mission . . . 4
- 1.2 The AI Policy – Read This First . . . 4
- 1.3 Box Color Summary . . . 5
- 1.4 Tools – Install Before Starting . . . 5

2 What You Are Building – FleetSync **6**
- 2.1 The Real-World Problem . . . 6
- 2.2 The Three Roles (RBAC) . . . 6
- 2.3 The Five Modules . . . 7
- 2.4 Database Entities and Relationships . . . 7
- 2.5 Complete API Endpoint List . . . 7
  - 2.5.1 Auth Endpoints . . . 7
  - 2.5.2 Vehicle Endpoints . . . 7
  - 2.5.3 Driver Endpoints . . . 8
  - 2.5.4 Trip Endpoints . . . 8

3 Week 1–2: Spring Boot Basics **9**
- 3.1 Week Goals . . . 9
- 3.2 Java Concepts to Refresh First . . . 9
- 3.3 Creating the Spring Boot Project . . . 10
- 3.4 Project Structure Explained . . . 10
- 3.5 The Entry Point: FleetsyncApplication.java . . . 11
- 3.6 Database Configuration . . . 12
- 3.7 Your First REST Controller . . . 12
- 3.8 Temporarily Disable Security for Testing . . . 13
- 3.9 Run and Test . . . 14
- 3.10 Git Setup – Day One . . . 15

4 Week 3–4: Database Layer (JPA, Entities, Repositories) **16**
- 4.1 Week Goals . . . 16
- 4.2 What is JPA and Hibernate? . . . 16
- 4.3 Package Structure – Create This Now . . . 16
- 4.4 Enums . . . 17
- 4.5 The User Entity . . . 18
- 4.6 The Vehicle Entity . . . 20
- 4.7 The Driver Entity . . . 21

4.8 The Trip Entity . . . 23
4.9 JPA Repositories . . . 24

5 Week 5–6: Service Layer and CRUD APIs 27
5.1 Week Goals . . . 27
5.2 Why DTOs? . . . 27
5.3 Vehicle DTOs . . . 27
5.4 Custom Exception Classes . . . 29
5.5 Global Exception Handler . . . 30
5.6 Vehicle Service . . . 31
5.7 Vehicle Controller . . . 34
5.8 Driver Module – Build Yourself . . . 36

6 Week 7: JWT Authentication 38
6.1 Week Goals . . . 38
6.2 How JWT Authentication Works . . . 38
6.3 Add JWT Dependency to pom.xml . . . 39
6.4 Auth DTOs . . . 39
6.5 JwtUtils – JWT Generation and Validation . . . 40
6.6 CustomUserDetailsService . . . 42
6.7 JwtAuthFilter . . . 43
6.8 Final Security Configuration . . . 45
6.9 Auth Service and Controller . . . 47

7 Week 8: Role-Based Access Control 50
7.1 Week Goals . . . 50
7.2 Applying Role Restrictions . . . 50
7.3 Restricting Drivers to Their Own Data . . . 51

8 Week 9: The Trip Module 53
8.1 Week Goals . . . 53
8.2 Trip DTOs . . . 53
8.3 Trip Service – The State Machine . . . 54

9 Week 10: Polish – Swagger, Validation, API Response 61
9.1 Standard API Response Wrapper . . . 61
9.2 Swagger / OpenAPI Documentation . . . 62

10 Week 11: Unit Testing 63
10.1 Why Testing Matters for Your Job Hunt . . . 63
10.2 VehicleService Unit Tests . . . 63

11 Week 12: Deployment and Portfolio 67
11.1 Dockerfile . . . 67
11.2 Environment-Safe application.properties . . . 67
11.3 Deploy to Railway (Free) . . . 68
11.4 Resume Bullet Point . . . 68

12 Interview Preparation – Top 20 Questions 69
12.1 Spring Boot . . . 69

12.2 Security . . . 69
12.3 JPA / Database . . . 70
12.4 General Java . . . 70
12.5 Pakistan and UAE Job Market Reality . . . 71

13 60-Day Execution Plan **72**
13.1 Job Search Parallel Track – Start Week 4 . . . 72

14 AI Development Guide **73**
14.1 The One Rule . . . 73
14.2 Complete AI Policy Table . . . 73
14.3 Effective AI Prompt Patterns . . . 73
14.3.1 Pattern 1: Explain Before Generate . . . 74
14.3.2 Pattern 2: Debug Specific Errors . . . 74
14.3.3 Pattern 3: Review Your Own Code . . . 74

15 Phase 2 – AI Integration (After the Job) **75**
15.1 The Architecture . . . 75
15.2 What the AI Adds . . . 75
15.3 Spring Boot Calling Python ML Service . . . 75

A Appendix A: Complete pom.xml **77**

B Appendix B: Postman Setup Guide **80**
B.1 Collection Structure . . . 80
B.2 Auto-Save JWT Token . . . 80

C Appendix C: Common Errors and Fixes **81**

Chapter 1

How to Use This Guide

1.1 The Mission

You have 60 days. You want a junior Java developer job or internship. You have IntelliJ Ultimate (student plan), a project idea, and the drive to execute. This guide converts that into a deployed, working project that passes screening calls.

Every chapter maps to one or two weeks of work. Every code block is explained line by line. Nothing is copy-paste-and-pray.

1.2 The AI Policy – Read This First

This is the most critical section in the entire guide.

ROBOT – AI CAN WRITE THIS

AI-Writable means: this is boilerplate, configuration, or repetitive structure. Senior devs also use templates/AI for this. You still **must read and understand every line** before moving on. Use GitHub Copilot, ChatGPT, or Claude to generate it. Then explain each line out loud as if teaching it to someone.

HAND – WRITE THIS YOURSELF (CRITICAL)

Write Yourself means: this is what interviewers test you on. If you cannot write this without AI, you will fail the technical screen. These are the logic, structure, and design decisions that prove you *understand* Spring Boot – not just that you can prompt an AI. Do not copy these. Write them yourself, make mistakes, fix them, and know why they work.

CONCEPT NOTE

Concept Notes explain the *why* behind the code. These become your interview answers to questions like "explain how JWT works" or "what is the difference between authentication and authorization?"

INTERVIEW GOLD

These contain exact talking points and common questions from UAE/Pakistan recruiters for this topic. Memorise these.

WATCH OUT

Common beginner mistakes. Read before writing the relevant code.

CHECKPOINT

Progress verification. Do not proceed to the next chapter until all boxes are checked.

1.3 Box Color Summary

Color	Type	Action Required
Purple left bar	AI Can Write	Generate then understand every line
Green left bar	Write Yourself	Write from scratch. Core interview material
Blue left bar	Concept Note	Theory behind the code. Interview answers
Orange left bar	Watch Out	Read before coding this section
Green left bar	Checkpoint	Verify before proceeding
Red left bar	Interview Gold	Memorise these talking points

1.4 Tools – Install Before Starting

1. **IntelliJ IDEA Ultimate** – Already installed (student plan). Update to latest.
2. **Java 17 (Temurin/OpenJDK)** – adoptium.net – verify: `java -version`
3. **XAMPP** – apachefriends.org – start the MySQL module
4. **Postman** – postman.com – for testing APIs
5. **Git** – git-scm.com – verify: `git -version`
6. **GitHub Account** – your portfolio home

CHECKPOINT

Run all three in your terminal before proceeding:

```
java -version     # openjdk version "17.x.x"
git --version     # git version 2.x.x
mysql --version   # start XAMPP first
```

All three must return version numbers with no errors.

Chapter 2

What You Are Building – FleetSync

2.1 The Real-World Problem

Companies like TCS Pakistan, Leopard Courier, Careem, and Aramex manage hundreds of vehicles daily. They need software to:

- Know which vehicle is assigned to which driver
- Know which driver is on which trip right now
- Let managers see everything; drivers see only their own data
- Track trip history for payroll and performance

This is a real problem – not a todo list. Recruiters know the difference.

2.2 The Three Roles (RBAC)

CONCEPT NOTE

RBAC – Role-Based Access Control. A security pattern where what you can do is determined by your *role*, not by who you are individually. In FleetSync your role is ADMIN, MANAGER, or DRIVER. Each role has a defined set of permissions. Spring Security implements RBAC via `@PreAuthorize` annotations.

Role	Permissions
ADMIN	Create user accounts, assign roles, deactivate users
MANAGER	Full CRUD on vehicles and drivers; create and manage trips; view all dashboards
DRIVER	View own trips only; mark trips started/completed; view own profile

2.3 The Five Modules

Module	Week Built	Responsibility
Auth	7–8	Register, login, JWT token, role assignment
Fleet / Vehicle	5–6	CRUD for all vehicle types
Driver	5–6	CRUD for drivers, soft delete
Trip	9	Create trips, assign driver+vehicle, lifecycle
User	8	Admin-level user management

2.4 Database Entities and Relationships

CONCEPT NOTE

Before writing a single line of code, understand how the data connects. This is called a **Database Schema**. The relationships below tell JPA how to create foreign keys automatically.

Entity	Key Fields	Relationship
User	id, username, email, password, role	Central auth entity
Driver	id, firstName, lastName, licenseNumber, user_id	OneToOne with User
Vehicle	id, plateNumber, type, model, status	Standalone
Trip	id, origin, destination, status, driver_id, vehicle_id	ManyToOne to Driver and Vehicle

2.5 Complete API Endpoint List

2.5.1 Auth Endpoints

Method	Endpoint	Access
POST	`/api/auth/register`	Public
POST	`/api/auth/login`	Public

2.5.2 Vehicle Endpoints

Method	Endpoint	Access
GET	`/api/vehicles`	MANAGER, ADMIN
GET	`/api/vehicles/{id}`	All roles
POST	`/api/vehicles`	MANAGER
PUT	`/api/vehicles/{id}`	MANAGER
DELETE	`/api/vehicles/{id}`	MANAGER
GET	`/api/vehicles/type/{type}`	MANAGER

2.5.3 Driver Endpoints

Method	Endpoint	Access
GET	`/api/drivers`	MANAGER
GET	`/api/drivers/{id}`	MANAGER
POST	`/api/drivers`	MANAGER
PUT	`/api/drivers/{id}`	MANAGER
DELETE	`/api/drivers/{id}`	MANAGER (soft delete)

2.5.4 Trip Endpoints

Method	Endpoint	Access
GET	`/api/trips`	MANAGER
GET	`/api/trips/{id}`	All roles
POST	`/api/trips`	MANAGER
PUT	`/api/trips/{id}/start`	MANAGER, DRIVER
PUT	`/api/trips/{id}/complete`	MANAGER, DRIVER
PUT	`/api/trips/{id}/cancel`	MANAGER
GET	`/api/trips/my-trips`	DRIVER only

Chapter 3

Week 1–2: Spring Boot Basics

3.1 Week Goals

By end of Week 2 you will have:

- A running Spring Boot project responding to HTTP
- A working `/api/test/hello` endpoint in Postman
- Understanding of how Spring Boot starts and why
- First Git commit pushed to GitHub

3.2 Java Concepts to Refresh First

HAND – WRITE THIS YOURSELF (CRITICAL)

Do not use AI for these. Interviewers test these directly. If any are shaky, spend one day on them before touching Spring Boot.

- **Classes and Objects** – instantiation, constructors
- **Interfaces** – `implements`, why they exist
- **Inheritance** – `extends`, method overriding, `super`
- **Access Modifiers** – `public`, `private`, `protected`
- **Collections** – `List`, `ArrayList`, `HashMap`
- **Exception Handling** – `try-catch-finally`, checked vs unchecked
- **Generics** – `List<String>`, `Optional<T>`

INTERVIEW GOLD

Most common junior question: "What is the difference between ArrayList and LinkedList?"

Answer: ArrayList is backed by a dynamic array. Random access is O(1). Insertion/deletion in the middle is O(n) because elements shift. LinkedList is a doubly-linked chain of nodes. Access is O(n) but insertion/deletion at known positions is O(1). For most Spring Boot use cases (iterating a DB result list), ArrayList is preferred.

3.3 Creating the Spring Boot Project

ROBOT – AI CAN WRITE THIS

Go to start.spring.io. This tool generates the project skeleton. Senior devs use this too – nobody hand-writes Maven pom.xml from scratch.

```
Project:        Maven
Language:       Java
Spring Boot:    3.4.x  (latest stable 3.x)
Packaging:      Jar
Java:           17

Group:          com.fleetsync
Artifact:       fleetsync
Name:           fleetsync
Package name:   com.fleetsync

Dependencies to add:
  Spring Web
  Spring Data JPA
  MySQL Driver
  Lombok
  Spring Security
  Spring Boot DevTools
  Validation
```

Listing 3.1: Spring Initializr Settings

Click **Generate**, download the zip, extract it, open in IntelliJ.

3.4 Project Structure Explained

```
fleetsync/
 src/
   main/
     java/com/fleetsync/
       FleetsyncApplication.java     <- Entry point
     resources/
       application.properties        <- Config file
 test/
   java/com/fleetsync/
     FleetsyncApplicationTests.java
```

```
pom.xml                                <- Maven build + dependencies
.gitignore
```

Listing 3.2: Generated Project Structure

CONCEPT NOTE

Maven is a build tool for Java (like npm for Node.js). Your `pom.xml` is like `package.json` – it declares dependencies and how to build the project. IntelliJ reads `pom.xml` on open and auto-downloads all declared libraries from Maven Central.

3.5 The Entry Point: FleetsyncApplication.java

HAND – WRITE THIS YOURSELF (CRITICAL)

Open `FleetsyncApplication.java`. Read it before writing anything else.

```java
package com.fleetsync;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication   // (1) Three annotations in one
public class FleetsyncApplication {

    public static void main(String[] args) {
        SpringApplication.run(FleetsyncApplication.class, args); // (2)
    }
}
```

Listing 3.3: FleetsyncApplication.java

Line-by-line:

(1) `@SpringBootApplication` is actually three annotations combined:

- `@Configuration` – this class can define Spring beans
- `@EnableAutoConfiguration` – Spring auto-configures based on jars present (sees MySQL driver, sets up DB connection automatically)
- `@ComponentScan` – scans all classes in `com.fleetsync` and sub-packages for annotations like `@Service`, `@RestController`, etc.

(2) `SpringApplication.run()` starts an embedded Tomcat web server (Spring Boot includes the web server – no need to deploy to external Tomcat).

CONCEPT NOTE

What is a Java Annotation? An annotation starts with `@`. It is metadata attached

to a class, method, or field that tells the framework (Spring) how to treat that element. When you write `@RestController`, you tell Spring: "this class handles HTTP requests, register it." Without the annotation, Spring ignores the class completely.

3.6 Database Configuration

ROBOT – AI CAN WRITE THIS

This is configuration. Use the block below exactly. Understand each property from the comments.

Open `src/main/resources/application.properties`:

```
spring.application.name=fleetsync

# Database
spring.datasource.url=jdbc:mysql://localhost:3306/fleetsync_db\
?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate
# create = drop + recreate tables on restart (use in early dev)
# update = update schema without dropping (use in mid dev)
# none   = do nothing (use in production)
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect\
.MySQL8Dialect

# Server
server.port=8080

# JWT (added later, prepare now)
jwt.secret=fleetsync-super-secret-key-minimum-32-chars-long!!
jwt.expiration=86400000
```

Listing 3.4: application.properties

Create the database in XAMPP phpMyAdmin or MySQL terminal:

```
CREATE DATABASE fleetsync_db;
-- Do NOT create tables manually.
-- JPA/Hibernate creates them automatically from your @Entity
   classes.
```

Listing 3.5: Create the database

3.7 Your First REST Controller

HAND – WRITE THIS YOURSELF (CRITICAL)

Write this yourself. This is the most fundamental Spring Boot pattern. You will write this structure 15 more times in this project. Build the muscle memory now.

Create package `com.fleetsync.controller` then create `TestController.java`:

```java
package com.fleetsync.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController                    // (1) handles HTTP, returns JSON/text
@RequestMapping("/api/test")       // (2) all methods start with /api/test
public class TestController {

    @GetMapping("/hello")          // (3) GET /api/test/hello
    public String hello() {
        return "Hello FleetSync! Spring Boot is running.";
    }

    @GetMapping("/status")         // GET /api/test/status
    public String status() {
        return "FleetSync API Status: UP";
    }
}
```

Listing 3.6: TestController.java

(1) `@RestController` = `@Controller` + `@ResponseBody`. The return value of each method is written directly to the HTTP response body (serialized to JSON or plain text). Without `@ResponseBody`, Spring would try to find a view template to render – wrong for REST APIs.

(2) Sets the base URL prefix for all methods in this class.

(3) Maps HTTP GET to this method. Full URL: `http://localhost:8080/api/test/hello`

3.8 Temporarily Disable Security for Testing

WATCH OUT

If you run now and call the endpoint in Postman, you get **401 Unauthorized**. Why? Because you added Spring Security as a dependency. By default it blocks everything. We will temporarily open all routes for development, then lock them down properly in Week 7.

ROBOT – AI CAN WRITE THIS

Create `com.fleetsync.config.SecurityConfig.java`. This is temporary boilerplate.

```java
package com.fleetsync.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
            throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .anyRequest().permitAll()  // DEVELOPMENT ONLY - lock down in Week 7
            );
        return http.build();
    }
}
```

Listing 3.7: SecurityConfig.java - Temporary open config for development

WATCH OUT

`.anyRequest().permitAll()` = zero security. This is ONLY for development. In Week 7 you will replace this with proper JWT security. Never deploy with this config.

3.9 Run and Test

1. In IntelliJ: press **Shift+F10** or click the green Run button
2. Console should show: `Tomcat started on port(s): 8080`
3. Open Postman: GET `http://localhost:8080/api/test/hello`
4. You should see: `Hello FleetSync! Spring Boot is running.`

3.10 Git Setup – Day One

```
git init
git add .
git commit -m "feat: initial Spring Boot project setup"

# Create repo on GitHub (name it fleetsync), then:
git remote add origin https://github.com/YOUR_USERNAME/
    fleetsync.git
git branch -M main
git push -u origin main
```

Listing 3.8: Initialize Git repository

CONCEPT NOTE

Conventional Commits – the industry standard commit message format:

- `feat:` – new feature added
- `fix:` – bug fixed
- `refactor:` – restructure, no logic change
- `docs:` – documentation update
- `chore:` – config, build, tooling

Recruiters check GitHub commit history. Clean commits look professional.

CHECKPOINT

Week 1–2 Checkpoint:

Spring Boot app starts with no errors

`/api/test/hello` returns response in Postman

`/api/test/status` returns response in Postman

Project pushed to GitHub with clean commit message

Chapter 4

Week 3–4: Database Layer (JPA, Entities, Repositories)

4.1 Week Goals

By end of Week 4:

- All entity classes defined and mapped to MySQL tables
- JPA auto-creating your database tables on startup
- Repository interfaces for each entity
- Understanding of JPA relationships (OneToMany, ManyToOne)

4.2 What is JPA and Hibernate?

CONCEPT NOTE

JPA (Java Persistence API) is a specification – a contract defining how Java objects map to relational databases. It is not a library itself.

Hibernate is the most popular *implementation* of JPA. When you use `@Entity`, `@Table`, `@OneToMany`, etc. you are using JPA annotations, but Hibernate is the engine actually talking to MySQL.

ORM (Object Relational Mapping): Instead of writing `INSERT INTO vehicles VALUES (...)`, you write `vehicleRepository.save(vehicle)` and JPA/Hibernate generates the SQL automatically.

4.3 Package Structure – Create This Now

```
com.fleetsync/
 config/        Security, JWT, CORS config
 controller/    REST controllers (@RestController)
 service/       Business logic (@Service)
 repository/    Database access (@Repository)
```

```
entity/        Database tables (@Entity)
dto/           Data Transfer Objects (request/response shapes)
exception/     Custom exceptions and global handler
security/      JWT filter, UserDetails implementation
enums/         Enum types (VehicleType, TripStatus, Role)
```

Listing 4.1: FleetSync package structure

CONCEPT NOTE

Layered Architecture (N-Tier): Each package is a layer with one job:

- **Controller** – receives HTTP request, validates input, calls service, returns response
- **Service** – contains business logic, calls repository, throws business exceptions
- **Repository** – talks to the database only, no business logic here
- **Entity** – represents database tables as Java objects

This is what every professional Java project uses. Separating concerns makes code maintainable and testable.

4.4 Enums

HAND – WRITE THIS YOURSELF (CRITICAL)

Write all enums yourself. They define the business domain language. Simple but important.

```java
package com.fleetsync.enums;

public enum VehicleType {
    TRUCK, BIKE, SHIP, TAXI, VAN
}
```

Listing 4.2: VehicleType.java

```java
package com.fleetsync.enums;

public enum VehicleStatus {
    AVAILABLE,     // Ready to be assigned
    ON_TRIP,       // Currently on a trip
    MAINTENANCE,   // Under repair
    RETIRED        // No longer in service
}
```

Listing 4.3: VehicleStatus.java

```java
package com.fleetsync.enums;

public enum TripStatus {
```

```
    PENDING,        // Created, driver not started
    IN_PROGRESS,    // Driver started the trip
    COMPLETED,      // Trip finished successfully
    CANCELLED       // Trip was cancelled
}
```

Listing 4.4: TripStatus.java

```
package com.fleetsync.enums;

public enum Role {
    ADMIN, MANAGER, DRIVER
}
```

Listing 4.5: Role.java

4.5 The User Entity

HAND – WRITE THIS YOURSELF (CRITICAL)

Write this yourself. The User entity is the center of your entire auth system. Every annotation here will be asked about in interviews. Understand what each one does before writing the next line.

```
package com.fleetsync.entity;

import com.fleetsync.enums.Role;
import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;

@Entity                                 // (1) maps class to DB table
@Table(name = "users")                  // (2) table name (avoids SQL keyword 'user')
@Data                                   // (3) Lombok: generates getters/setters/etc
@Builder                                // (4) Lombok: builder pattern
@NoArgsConstructor                      // (5) required by JPA
@AllArgsConstructor                     // (6) all-args constructor
public class User {

    @Id                                 // (7) primary key
    @GeneratedValue(strategy = GenerationType.IDENTITY) // (8) auto-increment
    private Long id;

    @Column(unique = true, nullable = false, length = 50)
    private String username;
```

```java

    @Column(unique = true, nullable = false)
    private String email;

    @Column(nullable = false)
    private String password;          // ALWAYS store hashed, never plain text

    @Enumerated(EnumType.STRING)      // (9) store as "MANAGER" not 0/1/2
    @Column(nullable = false)
    private Role role;

    @Column(nullable = false)
    private Boolean isActive = true;

    @Column(updatable = false)        // (10) set once, never updated
    private LocalDateTime createdAt;

    private LocalDateTime updatedAt;

    @PrePersist                       // (11) auto-runs before INSERT
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = LocalDateTime.now();
    }

    @PreUpdate                        // (12) auto-runs before UPDATE
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }
}
```

Listing 4.6: User.java

Key annotations explained:

(1) @Entity – Hibernate sees this and creates/maps a DB table.

(2) @Table(name = "users") – Without this, the table defaults to the class name. Since `user` is a reserved SQL keyword in MySQL, we specify `users`.

(3) @Data – Lombok generates all boilerplate: getters, setters, equals(), hashCode(), toString(). Without Lombok you would write 100+ lines manually.

(5) @NoArgsConstructor – JPA requires an empty constructor to instantiate entities when reading from DB.

(8) @GeneratedValue(IDENTITY) – Database handles auto-increment. When you call `save()`, DB assigns the next ID.

(9) @Enumerated(EnumType.STRING) – Without this, JPA stores enums as numbers (0, 1, 2). If you add a new enum value later, the numbers shift and your existing data is corrupted. Always use STRING.

(11) @PrePersist / @PreUpdate – JPA lifecycle hooks. These run automatically. You never need to manually set `createdAt` anywhere in your service code.

> **INTERVIEW GOLD**
>
> **"Why do we hash passwords and not store them plain?"**
>
> If the database is compromised (SQL injection, server breach), attackers get all passwords. If hashed with BCrypt, the hash cannot be reversed. BCrypt also uses a random *salt* – two users with the same password have different hashes, defeating rainbow table attacks. Spring Security's `BCryptPasswordEncoder` handles all this automatically.

4.6 The Vehicle Entity

```java
package com.fleetsync.entity;

import com.fleetsync.enums.VehicleStatus;
import com.fleetsync.enums.VehicleType;
import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;
import java.util.List;

@Entity
@Table(name = "vehicles")
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Vehicle {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false, length = 20)
    private String plateNumber;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private VehicleType type;

    @Column(nullable = false)
    private String model;

    @Column(nullable = false)
```

```
    private Integer capacity;

    @Enumerated(EnumType.STRING)
    @Builder.Default
    private VehicleStatus status = VehicleStatus.AVAILABLE;

    // One vehicle can have many trips
    @OneToMany(mappedBy = "vehicle",
               cascade = CascadeType.ALL,
               fetch = FetchType.LAZY)    // (1) don't load trips unless needed
    private List<Trip> trips;

    @Column(updatable = false)
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() { updatedAt = LocalDateTime.now(); }
}
```

Listing 4.7: Vehicle.java

CONCEPT NOTE

FetchType.LAZY vs EAGER:

LAZY (default for `@OneToMany`): When you load a Vehicle, its trips list is NOT loaded. It only loads when you call `vehicle.getTrips()`. Efficient – why load 1000 trips just to show a vehicle's plate number?

EAGER (default for `@ManyToOne`): Related entities load immediately with the parent.

N+1 Problem: Load 100 vehicles with EAGER trips = 1 query for vehicles + 100 queries for trips = 101 queries total. This is the N+1 problem. Fix: use a JOIN FETCH JPQL query. Very common interview topic.

4.7 The Driver Entity

```
package com.fleetsync.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDate;
```

```java
import java.time.LocalDateTime;
import java.util.List;

@Entity
@Table(name = "drivers")
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Driver {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String firstName;

    @Column(nullable = false)
    private String lastName;

    @Column(unique = true, nullable = false)
    private String licenseNumber;

    @Column(nullable = false)
    private String phoneNumber;

    private LocalDate licenseExpiry;

    @Column(nullable = false)
    private Boolean isActive = true;

    // Driver has one User account for login
    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", unique = true)  // FK column
        in drivers table
    private User user;

    // One driver can have many trips
    @OneToMany(mappedBy = "driver",
               cascade = CascadeType.ALL,
               fetch = FetchType.LAZY)
    private List<Trip> trips;

    @Column(updatable = false)
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
```

```
        updatedAt = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() { updatedAt = LocalDateTime.now()
        ; }
}
```

Listing 4.8: Driver.java

4.8 The Trip Entity

```
package com.fleetsync.entity;

import com.fleetsync.enums.TripStatus;
import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "trips")
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Trip {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String origin;

    @Column(nullable = false)
    private String destination;

    @Enumerated(EnumType.STRING)
    @Builder.Default
    private TripStatus status = TripStatus.PENDING;

    private String notes;

    // Many trips -> one driver
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "driver_id")
    private Driver driver;

    // Many trips -> one vehicle
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "vehicle_id")
```

```
    private Vehicle vehicle;

    // Manager who created this trip
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "created_by")
    private User createdBy;

    @Column(updatable = false)
    private LocalDateTime createdAt;
    private LocalDateTime startedAt;
    private LocalDateTime completedAt;

    @PrePersist
    protected void onCreate() { createdAt = LocalDateTime.now()
        ; }
}
```

Listing 4.9: Trip.java

4.9 JPA Repositories

CONCEPT NOTE

Spring Data JPA Repository: You define an interface extending `JpaRepository<Entity, ID>` and Spring generates all SQL at runtime automatically. You get `save()`, `findById()`, `findAll()`, `delete()` for free. Custom queries follow a naming convention – Spring parses the method name and generates SQL.

```
package com.fleetsync.repository;

import com.fleetsync.entity.User;
import com.fleetsync.enums.Role;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;
import java.util.Optional;

@Repository
public interface UserRepository extends JpaRepository<User,
    Long> {
    // Spring generates: SELECT * FROM users WHERE username = ?
    Optional<User> findByUsername(String username);

    Optional<User> findByEmail(String email);

    List<User> findByRole(Role role);

    boolean existsByUsername(String username);

    boolean existsByEmail(String email);
```

```
}
```

Listing 4.10: UserRepository.java

```
package com.fleetsync.repository;

import com.fleetsync.entity.Vehicle;
import com.fleetsync.enums.VehicleStatus;
import com.fleetsync.enums.VehicleType;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;
import java.util.List;
import java.util.Optional;

@Repository
public interface VehicleRepository extends JpaRepository<
    Vehicle, Long> {
    Optional<Vehicle> findByPlateNumber(String plateNumber);
    List<Vehicle> findByType(VehicleType type);
    List<Vehicle> findByStatus(VehicleStatus status);
    boolean existsByPlateNumber(String plateNumber);

    @Query("SELECT v FROM Vehicle v WHERE v.status = 'AVAILABLE
        ')
    List<Vehicle> findAllAvailable();
}
```

Listing 4.11: VehicleRepository.java

```
package com.fleetsync.repository;

import com.fleetsync.entity.Driver;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;
import java.util.List;
import java.util.Optional;

@Repository
public interface DriverRepository extends JpaRepository<Driver,
    Long> {
    Optional<Driver> findByLicenseNumber(String licenseNumber);
    Optional<Driver> findByUserId(Long userId);
    List<Driver> findByIsActive(Boolean isActive);
    boolean existsByLicenseNumber(String licenseNumber);

    @Query("SELECT d FROM Driver d WHERE d.isActive = true " +
           "AND d.id NOT IN " +
           "(SELECT t.driver.id FROM Trip t WHERE t.status = '
              IN_PROGRESS')")
    List<Driver> findAvailableDrivers();
```

```
}
```

Listing 4.12: DriverRepository.java

```java
package com.fleetsync.repository;

import com.fleetsync.entity.Trip;
import com.fleetsync.enums.TripStatus;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface TripRepository extends JpaRepository<Trip,
    Long> {
    List<Trip> findByStatus(TripStatus status);
    List<Trip> findByDriverId(Long driverId);
    List<Trip> findByVehicleId(Long vehicleId);
    List<Trip> findByDriverIdAndStatus(Long driverId,
        TripStatus status);
}
```

Listing 4.13: TripRepository.java

CHECKPOINT

Week 3–4 Checkpoint:

- App runs with no errors
- Open phpMyAdmin: 4 tables auto-created (users, vehicles, drivers, trips)
- Columns match your entity fields
- You can explain @OneToMany vs @ManyToOne
- You can explain LAZY vs EAGER loading

Chapter 5

Week 5–6: Service Layer and CRUD APIs

5.1 Week Goals

By end of Week 6:

- DTOs for all entities
- Service classes with full business logic
- Controller classes with all CRUD endpoints
- All APIs testable in Postman

5.2 Why DTOs?

CONCEPT NOTE

DTO (Data Transfer Object): A DTO is a plain class that carries data between layers. You *never* expose your `@Entity` directly in API responses. Reasons:

1. User entity has `password` – if you return the entity directly, the password goes to the client.
2. Circular references: Vehicle has List<Trip>, Trip has Vehicle – this causes infinite recursion when Jackson serializes to JSON.
3. You may want to shape the response differently from the DB structure.

Pattern: Client sends **RequestDTO** → Controller → Service converts to Entity → Repository saves → Service converts Entity to **ResponseDTO** → Controller returns ResponseDTO.

5.3 Vehicle DTOs

HAND – WRITE THIS YOURSELF (CRITICAL)

Write the RequestDTO yourself. This pattern repeats for every entity. Once you understand it here, apply it to Driver and Trip independently.

```java
package com.fleetsync.dto;

import com.fleetsync.enums.VehicleType;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import lombok.Data;

@Data
public class VehicleRequestDto {

    @NotBlank(message = "Plate number is required")
    private String plateNumber;

    @NotNull(message = "Vehicle type is required")
    private VehicleType type;

    @NotBlank(message = "Model is required")
    private String model;

    @NotNull(message = "Capacity is required")
    @Positive(message = "Capacity must be a positive number")
    private Integer capacity;

    // status is NOT here. New vehicles are always AVAILABLE.
    // Client should not control status on creation.
}
```

Listing 5.1: VehicleRequestDto.java

```java
package com.fleetsync.dto;

import com.fleetsync.enums.VehicleStatus;
import com.fleetsync.enums.VehicleType;
import lombok.Builder;
import lombok.Data;
import java.time.LocalDateTime;

@Data
@Builder
public class VehicleResponseDto {
    private Long id;
    private String plateNumber;
    private VehicleType type;
    private String model;
    private Integer capacity;
    private VehicleStatus status;
```

```java
    private LocalDateTime createdAt;

    // Trips list is NOT included. If caller wants trips, they call
    // /api/vehicles/{id}/trips separately. Keeps responses lean.
}
```

Listing 5.2: VehicleResponseDto.java

5.4 Custom Exception Classes

ROBOT – AI CAN WRITE THIS

Exception class bodies are boilerplate. Generate them. Understand what HTTP status each maps to.

```java
package com.fleetsync.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND)   // HTTP 404
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

Listing 5.3: ResourceNotFoundException.java

```java
package com.fleetsync.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.CONFLICT)    // HTTP 409
public class DuplicateResourceException extends RuntimeException {
    public DuplicateResourceException(String message) {
        super(message);
    }
}
```

Listing 5.4: DuplicateResourceException.java

```java
package com.fleetsync.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;
```

```java

@ResponseStatus(HttpStatus.BAD_REQUEST) // HTTP 400
public class BusinessException extends RuntimeException {
    public BusinessException(String message) {
        super(message);
    }
}
```

Listing 5.5: BusinessException.java

5.5 Global Exception Handler

CONCEPT NOTE

Without a global handler, Spring returns ugly default error responses exposing your stack trace. With `@RestControllerAdvice`, you catch all exceptions in one place and return clean, consistent JSON.

```java
package com.fleetsync.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.
    MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler
    ;
import org.springframework.web.bind.annotation.
    RestControllerAdvice;
import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleNotFound(
            ResourceNotFoundException ex) {
        return build(HttpStatus.NOT_FOUND, ex.getMessage());
    }

    @ExceptionHandler(DuplicateResourceException.class)
    public ResponseEntity<Map<String, Object>> handleDuplicate(
            DuplicateResourceException ex) {
        return build(HttpStatus.CONFLICT, ex.getMessage());
    }

    @ExceptionHandler(BusinessException.class)
    public ResponseEntity<Map<String, Object>> handleBusiness(
            BusinessException ex) {
```

```java
        return build(HttpStatus.BAD_REQUEST, ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidation
        (
            MethodArgumentNotValidException ex) {
        Map<String, String> fieldErrors = new HashMap<>();
        ex.getBindingResult().getAllErrors().forEach(err -> {
            String field = ((FieldError) err).getField();
            fieldErrors.put(field, err.getDefaultMessage());
        });
        Map<String, Object> resp = new HashMap<>();
        resp.put("timestamp", LocalDateTime.now());
        resp.put("status", 400);
        resp.put("error", "Validation Failed");
        resp.put("details", fieldErrors);
        return ResponseEntity.badRequest().body(resp);
    }

    private ResponseEntity<Map<String, Object>> build(
            HttpStatus status, String message) {
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", status.value());
        body.put("error", status.getReasonPhrase());
        body.put("message", message);
        return ResponseEntity.status(status).body(body);
    }
}
```

Listing 5.6: GlobalExceptionHandler.java

5.6 Vehicle Service

HAND – WRITE THIS YOURSELF (CRITICAL)

The service layer is where your business logic lives. This is the most important layer to write yourself. Interviewers ask about this logic directly: "How do you prevent duplicate plate numbers?" "What happens if you try to delete a vehicle on a trip?"

```java
package com.fleetsync.service;

import com.fleetsync.dto.VehicleRequestDto;
import com.fleetsync.dto.VehicleResponseDto;
import com.fleetsync.entity.Vehicle;
import com.fleetsync.enums.VehicleStatus;
import com.fleetsync.enums.VehicleType;
import com.fleetsync.exception.BusinessException;
import com.fleetsync.exception.DuplicateResourceException;
```

```java
import com.fleetsync.exception.ResourceNotFoundException;
import com.fleetsync.repository.VehicleRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor  // Lombok: constructor injection for all final fields
public class VehicleService {

    private final VehicleRepository vehicleRepository;

    // -- CREATE ----------------------------------------------------------------
    @Transactional
    public VehicleResponseDto createVehicle(VehicleRequestDto req) {
        // Business rule: plate number must be unique
        if (vehicleRepository.existsByPlateNumber(req.getPlateNumber())) {
            throw new DuplicateResourceException(
                "Vehicle with plate " + req.getPlateNumber() + " already exists");
        }

        Vehicle vehicle = Vehicle.builder()
            .plateNumber(req.getPlateNumber().toUpperCase())
            .type(req.getType())
            .model(req.getModel())
            .capacity(req.getCapacity())
            .status(VehicleStatus.AVAILABLE)  // always AVAILABLE on creation
            .build();

        return toDto(vehicleRepository.save(vehicle));
    }

    // -- READ ALL ----------------------------------------------------------------
    public List<VehicleResponseDto> getAllVehicles() {
        return vehicleRepository.findAll()
            .stream().map(this::toDto).collect(Collectors.toList());
    }

```

```java
    // -- READ ONE ----------------------------------------------------------------
    public VehicleResponseDto getVehicleById(Long id) {
        return toDto(findOrThrow(id));
    }

    // -- READ BY TYPE ------------------------------------------------------------
    public List<VehicleResponseDto> getByType(VehicleType type) {
        return vehicleRepository.findByType(type)
            .stream().map(this::toDto).collect(Collectors.toList());
    }

    // -- UPDATE ------------------------------------------------------------------
    @Transactional
    public VehicleResponseDto updateVehicle(Long id, VehicleRequestDto req) {
        Vehicle vehicle = findOrThrow(id);

        String newPlate = req.getPlateNumber().toUpperCase();
        if (!vehicle.getPlateNumber().equals(newPlate)
                && vehicleRepository.existsByPlateNumber(newPlate)) {
            throw new DuplicateResourceException(
                "Plate number " + newPlate + " is already in use");
        }

        vehicle.setPlateNumber(newPlate);
        vehicle.setModel(req.getModel());
        vehicle.setCapacity(req.getCapacity());
        vehicle.setType(req.getType());

        return toDto(vehicleRepository.save(vehicle));
    }

    // -- DELETE ------------------------------------------------------------------
    @Transactional
    public void deleteVehicle(Long id) {
        Vehicle vehicle = findOrThrow(id);

        // Business rule: cannot delete a vehicle on a trip
        if (vehicle.getStatus() == VehicleStatus.ON_TRIP) {
```

```java
            throw new BusinessException(
                "Cannot delete vehicle currently ON_TRIP");
        }

        vehicleRepository.delete(vehicle);
    }

    // -- PRIVATE HELPERS ----------------------------------------------------
    private Vehicle findOrThrow(Long id) {
        return vehicleRepository.findById(id)
            .orElseThrow(() ->
                new ResourceNotFoundException("Vehicle not found: " + id));
    }

    private VehicleResponseDto toDto(Vehicle v) {
        return VehicleResponseDto.builder()
            .id(v.getId())
            .plateNumber(v.getPlateNumber())
            .type(v.getType())
            .model(v.getModel())
            .capacity(v.getCapacity())
            .status(v.getStatus())
            .createdAt(v.getCreatedAt())
            .build();
    }
}
```

Listing 5.7: VehicleService.java

CONCEPT NOTE

@Transactional wraps the method in a database transaction. If an exception is thrown mid-method, all DB changes in that method are rolled back. This is critical for data consistency – if you update a vehicle status AND a trip record, both must succeed or neither should.

@RequiredArgsConstructor (Lombok) generates a constructor for all `final` fields. This is **constructor injection** – the preferred dependency injection style in Spring Boot. Avoid field injection (`@Autowired` on fields) in production code.

5.7 Vehicle Controller

HAND – WRITE THIS YOURSELF (CRITICAL)

Write every `@PostMapping`, `@GetMapping` yourself. These are direct expressions of your API design. You must be able to write a CRUD controller in an interview.

```java
package com.fleetsync.controller;

```

```java
import com.fleetsync.dto.VehicleRequestDto;
import com.fleetsync.dto.VehicleResponseDto;
import com.fleetsync.enums.VehicleType;
import com.fleetsync.service.VehicleService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/vehicles")
@RequiredArgsConstructor
@CrossOrigin(origins = "*")
public class VehicleController {

    private final VehicleService vehicleService;

    @PostMapping
    public ResponseEntity<VehicleResponseDto> create(
            @Valid @RequestBody VehicleRequestDto req) {
        // @Valid triggers @NotBlank/@NotNull checks on the DTO
        // @RequestBody parses JSON body into VehicleRequestDto
        return ResponseEntity.status(HttpStatus.CREATED)
                             .body(vehicleService.createVehicle
                                (req));
    }

    @GetMapping
    public ResponseEntity<List<VehicleResponseDto>> getAll() {
        return ResponseEntity.ok(vehicleService.getAllVehicles
            ());
    }

    @GetMapping("/{id}")
    public ResponseEntity<VehicleResponseDto> getById(
            @PathVariable Long id) {
        return ResponseEntity.ok(vehicleService.getVehicleById(
            id));
    }

    @GetMapping("/type/{type}")
    public ResponseEntity<List<VehicleResponseDto>> getByType(
            @PathVariable VehicleType type) {
        return ResponseEntity.ok(vehicleService.getByType(type)
            );
    }

    @PutMapping("/{id}")
    public ResponseEntity<VehicleResponseDto> update(
```

```
            @PathVariable Long id,
            @Valid @RequestBody VehicleRequestDto req) {
        return ResponseEntity.ok(vehicleService.updateVehicle(id, req));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        vehicleService.deleteVehicle(id);
        return ResponseEntity.noContent().build();    // HTTP 204
    }
}
```

Listing 5.8: VehicleController.java

INTERVIEW GOLD

"What HTTP status codes do you use and why?"

Memorise these five:

- **200 OK** – successful GET or PUT
- **201 Created** – successful POST (resource was created)
- **204 No Content** – successful DELETE (nothing to return)
- **400 Bad Request** – validation error or bad input
- **401 Unauthorized** – no token or bad token
- **403 Forbidden** – valid token but wrong role
- **404 Not Found** – resource does not exist
- **409 Conflict** – duplicate resource (plate already exists)

5.8 Driver Module – Build Yourself

HAND – WRITE THIS YOURSELF (CRITICAL)

Now build the Driver module entirely on your own. Do not look at the Vehicle code while doing it. The structure is identical but the business rules differ.

Build these four files:

1. `DriverRequestDto` – fields: firstName, lastName, licenseNumber, phoneNumber, licenseExpiry
2. `DriverResponseDto` – id, firstName, lastName, licenseNumber, phoneNumber, isActive, createdAt
3. `DriverService` – createDriver, getAllDrivers, getDriverById, updateDriver, deactivateDriver

4. `DriverController` – all 5 endpoints

Business rules to enforce:

- License number must be unique
- Cannot create driver if licenseExpiry is in the past
- Use soft delete: `driver.setIsActive(false)` instead of `repository.delete()`

CONCEPT NOTE

Soft Delete vs Hard Delete:

Hard Delete: `DELETE FROM drivers WHERE id = ?` – row is gone forever. If a trip referenced this driver, you get a foreign key error or orphaned data.

Soft Delete: Set `isActive = false`. Row stays in DB. All queries filter by `isActive = true`. Historical records (trips, logs) remain intact. This is the industry standard for any entity with relationships.

CHECKPOINT

Week 5–6 Checkpoint in Postman:

POST `/api/vehicles` with JSON body – returns 201

GET `/api/vehicles` – returns list

GET `/api/vehicles/1` – returns single vehicle

PUT `/api/vehicles/1` – updates vehicle

DELETE `/api/vehicles/1` – returns 204

POST with duplicate plate – returns 409 with message

GET non-existent ID – returns 404 with message

Driver module: all 5 endpoints working

Chapter 6

Week 7: JWT Authentication

6.1 Week Goals

By end of Week 7:

- Working Register endpoint
- Working Login endpoint returning a JWT token
- All other endpoints protected – require valid token
- Full understanding of how JWT works end-to-end

6.2 How JWT Authentication Works

CONCEPT NOTE

JWT (JSON Web Token) – step by step:

Step 1 – Login: Client sends username + password to POST `/api/auth/login`. Server verifies credentials. If valid, it generates a JWT token and returns it. The server does NOT store this token anywhere.

Step 2 – Protected Request: For every subsequent request the client includes the token in the header: `Authorization: Bearer <token>`

Step 3 – Verification: The JWT filter intercepts every request, extracts the token, verifies its cryptographic signature, and if valid, sets the user's identity in Spring Security's SecurityContext.

Why stateless: The server never stores sessions. Each token is self-contained and carries the user's ID and role (called *claims*). The server only verifies the signature.

JWT Structure – three parts separated by dots:

- **Header:** algorithm used (HS256)
- **Payload:** claims (userId, role, expiry). Base64 encoded – NOT encrypted. Anyone can decode it.

- **Signature:** HMAC of (header + payload) using the server's secret key. Only the server can create a valid signature.

6.3 Add JWT Dependency to pom.xml

ROBOT – AI CAN WRITE THIS

Adding a library to pom.xml is configuration. Use this block exactly.

```
<!-- JWT -->
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.3</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.12.3</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.12.3</version>
    <scope>runtime</scope>
</dependency>
```

Listing 6.1: JWT dependencies in pom.xml

6.4 Auth DTOs

```
package com.fleetsync.dto;

import jakarta.validation.constraints.NotBlank;
import lombok.Data;

@Data
public class LoginRequestDto {
    @NotBlank(message = "Username is required")
    private String username;

    @NotBlank(message = "Password is required")
    private String password;
}
```

Listing 6.2: LoginRequestDto.java

```
package com.fleetsync.dto;

```

```java
import com.fleetsync.enums.Role;
import jakarta.validation.constraints.*;
import lombok.Data;

@Data
public class RegisterRequestDto {
    @NotBlank @Size(min = 3, max = 50)
    private String username;

    @NotBlank @Email
    private String email;

    @NotBlank @Size(min = 6, message = "Password min 6
       characters")
    private String password;

    @NotNull
    private Role role;
}
```

Listing 6.3: RegisterRequestDto.java

```java
package com.fleetsync.dto;

import com.fleetsync.enums.Role;
import lombok.Builder;
import lombok.Data;

@Data
@Builder
public class AuthResponseDto {
    private String token;
    private String tokenType;    // Always "Bearer"
    private String username;
    private Role role;
    private Long userId;
}
```

Listing 6.4: AuthResponseDto.java

6.5 JwtUtils – JWT Generation and Validation

HAND – WRITE THIS YOURSELF (CRITICAL)

This class is core security infrastructure. Understand every method. `generateToken`, `extractUsername`, and `isTokenValid` are asked about in interviews.

```java
package com.fleetsync.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
```

```java
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;
import javax.crypto.SecretKey;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

@Component
public class JwtUtils {

    @Value("${jwt.secret}")
    private String secretKey;

    @Value("${jwt.expiration}")
    private long jwtExpiration;

    public String generateToken(UserDetails userDetails) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("role",
            userDetails.getAuthorities().iterator().next().getAuthority());
        return Jwts.builder()
            .claims(claims)
            .subject(userDetails.getUsername())
            .issuedAt(new Date(System.currentTimeMillis()))
            .expiration(new Date(System.currentTimeMillis() + jwtExpiration))
            .signWith(getSigningKey())
            .compact();
    }

    public boolean isTokenValid(String token, UserDetails userDetails) {
        return extractUsername(token).equals(userDetails.getUsername())
            && !isTokenExpired(token);
    }

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    private boolean isTokenExpired(String token) {
        return extractClaim(token, Claims::getExpiration).before(new Date());
    }

```

```java
    public <T> T extractClaim(String token, Function<Claims, T> resolver) {
        Claims claims = Jwts.parser()
            .verifyWith(getSigningKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
        return resolver.apply(claims);
    }

    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(secretKey.getBytes());
    }
}
```

Listing 6.5: JwtUtils.java

6.6 CustomUserDetailsService

CONCEPT NOTE

`UserDetailsService` is a Spring Security interface with one method: `loadUserByUsername(String username)`. Spring Security calls this during authentication to load the user from DB. You must implement it.

```java
package com.fleetsync.security;

import com.fleetsync.entity.User;
import com.fleetsync.repository.UserRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.*;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username)
            throws UsernameNotFoundException {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() ->
                new UsernameNotFoundException("User not found: " + username));

```

```java
        return new org.springframework.security.core.
            userdetails.User(
            user.getUsername(),
            user.getPassword(),
            // Spring Security requires "ROLE_" prefix on
                authorities
            List.of(new SimpleGrantedAuthority("ROLE_" + user.
                getRole().name()))
        );
    }
}
```

Listing 6.6: CustomUserDetailsService.java

6.7 JwtAuthFilter

CONCEPT NOTE

Filter Chain: Every HTTP request passes through a chain of filters before reaching your controller. `JwtAuthFilter` is a custom filter you insert into this chain. It runs before every request, extracts the JWT from the Authorization header, validates it, and if valid, sets the authenticated user in the SecurityContext. Think of it as a security checkpoint.

```java
package com.fleetsync.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.springframework.security.authentication
        .UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.
    SecurityContextHolder;
import org.springframework.security.core.userdetails.
    UserDetails;
import org.springframework.security.web.authentication
        .WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
import java.io.IOException;

@Component
@RequiredArgsConstructor
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtUtils jwtUtils;
    private final CustomUserDetailsService userDetailsService;

```

```java
    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain chain)
            throws ServletException, IOException {

        // (1) Get Authorization header
        final String authHeader = request.getHeader("Authorization");

        // (2) If missing or wrong format, skip this filter
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            chain.doFilter(request, response);
            return;
        }

        // (3) Extract token (remove "Bearer " prefix = 7 chars)
        final String jwt = authHeader.substring(7);

        // (4) Extract username from token
        final String username = jwtUtils.extractUsername(jwt);

        // (5) If username valid and not already authenticated
        if (username != null
                && SecurityContextHolder.getContext()
                                        .getAuthentication() == null) {

            UserDetails userDetails =
                userDetailsService.loadUserByUsername(username);

            if (jwtUtils.isTokenValid(jwt, userDetails)) {
                // (6) Create auth token and set in SecurityContext
                var authToken =
                    new UsernamePasswordAuthenticationToken(
                        userDetails, null,
                        userDetails.getAuthorities());
                authToken.setDetails(
                    new WebAuthenticationDetailsSource()
                        .buildDetails(request));
                SecurityContextHolder.getContext()
                                     .setAuthentication(authToken);
            }
        }

```

```
        // (7) Continue to next filter / controller
        chain.doFilter(request, response);
    }
}
```

Listing 6.7: JwtAuthFilter.java

6.8 Final Security Configuration

Replace the temporary `SecurityConfig.java`:

```
package com.fleetsync.config;

import com.fleetsync.security.CustomUserDetailsService;
import com.fleetsync.security.JwtAuthFilter;
import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.
    AuthenticationManager;
import org.springframework.security.authentication.
    AuthenticationProvider;
import org.springframework.security.authentication.dao
        .DaoAuthenticationProvider;
import org.springframework.security.config.annotation.
    authentication
        .configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.
    configuration
        .EnableMethodSecurity;
import org.springframework.security.config.annotation.web.
    builders
        .HttpSecurity;
import org.springframework.security.config.annotation.web.
    configuration
        .EnableWebSecurity;
import org.springframework.security.config.http.
    SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.
    BCryptPasswordEncoder;
import org.springframework.security.crypto.password.
    PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication
        .UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
@RequiredArgsConstructor
public class SecurityConfig {
```

```java

    private final JwtAuthFilter jwtAuthFilter;
    private final CustomUserDetailsService userDetailsService;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
            throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/test/**").permitAll()
                .requestMatchers(
                    "/swagger-ui/**",
                    "/api-docs/**").permitAll()
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter,
                UsernamePasswordAuthenticationFilter.class)
            .authenticationProvider(authenticationProvider());
        return http.build();
    }

    @Bean
    public AuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider p = new DaoAuthenticationProvider();
        p.setUserDetailsService(userDetailsService);
        p.setPasswordEncoder(passwordEncoder());
        return p;
    }

    @Bean
    public AuthenticationManager authenticationManager(
            AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

Listing 6.8: SecurityConfig.java – Production JWT Security

6.9 Auth Service and Controller

```java
package com.fleetsync.service;

import com.fleetsync.dto.AuthResponseDto;
import com.fleetsync.dto.LoginRequestDto;
import com.fleetsync.dto.RegisterRequestDto;
import com.fleetsync.entity.User;
import com.fleetsync.exception.DuplicateResourceException;
import com.fleetsync.repository.UserRepository;
import com.fleetsync.security.CustomUserDetailsService;
import com.fleetsync.security.JwtUtils;
import lombok.RequiredArgsConstructor;
import org.springframework.security.authentication.
    AuthenticationManager;
import org.springframework.security.authentication
        .UsernamePasswordAuthenticationToken;
import org.springframework.security.core.userdetails.
    UserDetails;
import org.springframework.security.crypto.password.
    PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional
    ;

@Service
@RequiredArgsConstructor
public class AuthService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtUtils jwtUtils;
    private final AuthenticationManager authenticationManager;
    private final CustomUserDetailsService userDetailsService;

    @Transactional
    public AuthResponseDto register(RegisterRequestDto req) {
        if (userRepository.existsByUsername(req.getUsername()))
            throw new DuplicateResourceException(
                "Username taken: " + req.getUsername());
        if (userRepository.existsByEmail(req.getEmail()))
            throw new DuplicateResourceException(
                "Email already registered: " + req.getEmail());

        User user = User.builder()
            .username(req.getUsername())
            .email(req.getEmail())
            .password(passwordEncoder.encode(req.getPassword())
                )
            .role(req.getRole())
            .isActive(true)
```

```java
            .build();

        User saved = userRepository.save(user);
        UserDetails ud = userDetailsService.loadUserByUsername(
            saved.getUsername());

        return AuthResponseDto.builder()
            .token(jwtUtils.generateToken(ud))
            .tokenType("Bearer")
            .username(saved.getUsername())
            .role(saved.getRole())
            .userId(saved.getId())
            .build();
    }

    public AuthResponseDto login(LoginRequestDto req) {
        // Throws BadCredentialsException if wrong
        authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                req.getUsername(), req.getPassword()));

        UserDetails ud =
            userDetailsService.loadUserByUsername(req.
               getUsername());
        User user = userRepository.findByUsername(req.
           getUsername())
            .orElseThrow();

        return AuthResponseDto.builder()
            .token(jwtUtils.generateToken(ud))
            .tokenType("Bearer")
            .username(user.getUsername())
            .role(user.getRole())
            .userId(user.getId())
            .build();
    }
}
```

Listing 6.9: AuthService.java

```java
package com.fleetsync.controller;

import com.fleetsync.dto.AuthResponseDto;
import com.fleetsync.dto.LoginRequestDto;
import com.fleetsync.dto.RegisterRequestDto;
import com.fleetsync.service.AuthService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

```

```java
@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
@CrossOrigin(origins = "*")
public class AuthController {

    private final AuthService authService;

    @PostMapping("/register")
    public ResponseEntity<AuthResponseDto> register(
            @Valid @RequestBody RegisterRequestDto req) {
        return ResponseEntity.status(HttpStatus.CREATED)
                                .body(authService.register(req));
    }

    @PostMapping("/login")
    public ResponseEntity<AuthResponseDto> login(
            @Valid @RequestBody LoginRequestDto req) {
        return ResponseEntity.ok(authService.login(req));
    }
}
```

Listing 6.10: AuthController.java

CHECKPOINT

Week 7 Checkpoint:

POST `/api/auth/register` – returns 201 with token

POST `/api/auth/login` – returns token

GET `/api/vehicles` without token – returns 401

GET `/api/vehicles` with `Authorization: Bearer <token>` – returns 200

You can explain JWT end-to-end without notes

Chapter 7

Week 8: Role-Based Access Control

7.1 Week Goals

By end of Week 8:

- `@PreAuthorize` on all endpoints enforcing role restrictions
- MANAGER cannot access ADMIN endpoints and vice versa
- DRIVER can only access own data

7.2 Applying Role Restrictions

With `@EnableMethodSecurity(prePostEnabled = true)` in SecurityConfig, you can now use `@PreAuthorize` on any controller or service method.

> **HAND – WRITE THIS YOURSELF (CRITICAL)**
>
> Add `@PreAuthorize` annotations to all existing controllers. Security code – write and test yourself.

```java
@PostMapping
@PreAuthorize("hasRole('MANAGER')")
public ResponseEntity<VehicleResponseDto> create(
        @Valid @RequestBody VehicleRequestDto req) {
    return ResponseEntity.status(HttpStatus.CREATED)
                         .body(vehicleService.createVehicle(req
                            ));
}

@GetMapping
@PreAuthorize("hasAnyRole('MANAGER', 'ADMIN')")
public ResponseEntity<List<VehicleResponseDto>> getAll() {
    return ResponseEntity.ok(vehicleService.getAllVehicles());
}

@GetMapping("/{id}")
```

```java
@PreAuthorize("hasAnyRole('MANAGER', 'ADMIN', 'DRIVER')")
public ResponseEntity<VehicleResponseDto> getById(
        @PathVariable Long id) {
    return ResponseEntity.ok(vehicleService.getVehicleById(id));
}

@PutMapping("/{id}")
@PreAuthorize("hasRole('MANAGER')")
public ResponseEntity<VehicleResponseDto> update(
        @PathVariable Long id,
        @Valid @RequestBody VehicleRequestDto req) {
    return ResponseEntity.ok(vehicleService.updateVehicle(id, req));
}

@DeleteMapping("/{id}")
@PreAuthorize("hasRole('MANAGER')")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    vehicleService.deleteVehicle(id);
    return ResponseEntity.noContent().build();
}
```

Listing 7.1: VehicleController.java – updated with RBAC

7.3 Restricting Drivers to Their Own Data

```java
// Add this private helper to any Service class:
private String getCurrentUsername() {
    return SecurityContextHolder.getContext()
                                .getAuthentication().getName();
}
```

Listing 7.2: Getting the current user inside a Service

```java
public List<TripResponseDto> getMyTrips() {
    String username = getCurrentUsername();

    User user = userRepository.findByUsername(username)
        .orElseThrow(() ->
            new ResourceNotFoundException("User not found"));

    Driver driver = driverRepository.findByUserId(user.getId())
        .orElseThrow(() ->
            new ResourceNotFoundException("Driver profile not found"));

    return tripRepository.findByDriverId(driver.getId())
        .stream().map(this::toDto).collect(Collectors.toList());
}
```

Listing 7.3: TripService – getMyTrips for DRIVER role

INTERVIEW GOLD

"What is the difference between Authentication and Authorization?"

Authentication = verifying *who you are*. You prove identity with credentials. Spring Security's `AuthenticationManager` handles this. JWT is proof of previous authentication.

Authorization = determining *what you can do* after identity is confirmed. `@PreAuthorize` and the SecurityFilterChain handle this based on roles.

Analogy: Authentication is showing your ID at the door. Authorization is the bouncer checking whether your ticket is VIP or general.

Chapter 8

Week 9: The Trip Module

8.1 Week Goals

Trip is the most complex and business-logic-heavy part. By end of Week 9:

- Full Trip CRUD
- State machine: PENDING → IN_PROGRESS → COMPLETED
- Vehicle status auto-updates when trip starts/ends
- Driver self-service endpoints

8.2 Trip DTOs

```java
package com.fleetsync.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.Data;

@Data
public class TripRequestDto {
    @NotBlank private String origin;
    @NotBlank private String destination;
    @NotNull  private Long driverId;
    @NotNull  private Long vehicleId;
    private String notes;
}
```

Listing 8.1: TripRequestDto.java

```java
package com.fleetsync.dto;

import com.fleetsync.enums.TripStatus;
import lombok.Builder;
import lombok.Data;
import java.time.LocalDateTime;
```

```java

@Data
@Builder
public class TripResponseDto {
    private Long id;
    private String origin;
    private String destination;
    private TripStatus status;
    private String notes;
    private Long driverId;
    private String driverName;
    private Long vehicleId;
    private String vehiclePlate;
    private String vehicleType;
    private LocalDateTime createdAt;
    private LocalDateTime startedAt;
    private LocalDateTime completedAt;
}
```

Listing 8.2: TripResponseDto.java

8.3 Trip Service – The State Machine

HAND – WRITE THIS YOURSELF (CRITICAL)

This is the most important service you will write. The state machine (how trip status transitions work) is exactly what interviewers test. Write every method yourself.

```java
package com.fleetsync.service;

import com.fleetsync.dto.TripRequestDto;
import com.fleetsync.dto.TripResponseDto;
import com.fleetsync.entity.*;
import com.fleetsync.enums.TripStatus;
import com.fleetsync.enums.VehicleStatus;
import com.fleetsync.exception.BusinessException;
import com.fleetsync.exception.ResourceNotFoundException;
import com.fleetsync.repository.*;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.context.
    SecurityContextHolder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional
    ;
import java.time.LocalDateTime;
import java.util.List;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
```

```java
public class TripService {

    private final TripRepository tripRepository;
    private final DriverRepository driverRepository;
    private final VehicleRepository vehicleRepository;
    private final UserRepository userRepository;

    // -- CREATE ----------------------------------------------------------------

    @Transactional
    public TripResponseDto createTrip(TripRequestDto req) {

        Driver driver = driverRepository.findById(req.getDriverId())
            .orElseThrow(() -> new ResourceNotFoundException(
                "Driver not found: " + req.getDriverId()));

        if (!driver.getIsActive())
            throw new BusinessException(
                "Cannot assign trip to inactive driver");

        Vehicle vehicle = vehicleRepository.findById(req.getVehicleId())
            .orElseThrow(() -> new ResourceNotFoundException(
                "Vehicle not found: " + req.getVehicleId()));

        if (vehicle.getStatus() != VehicleStatus.AVAILABLE)
            throw new BusinessException(
                "Vehicle " + vehicle.getPlateNumber()
                + " is not available. Status: " + vehicle.getStatus());

        String username = SecurityContextHolder.getContext()
            .getAuthentication().getName();
        User manager = userRepository.findByUsername(username)
            .orElseThrow();

        Trip trip = Trip.builder()
            .origin(req.getOrigin())
            .destination(req.getDestination())
            .driver(driver)
            .vehicle(vehicle)
            .createdBy(manager)
            .notes(req.getNotes())
            .status(TripStatus.PENDING)
            .build();

        return toDto(tripRepository.save(trip));
    }

```

```java
    // -- START TRIP (PENDING -> IN_PROGRESS) ----------------------------------
    @Transactional
    public TripResponseDto startTrip(Long id) {
        Trip trip = findOrThrow(id);

        if (trip.getStatus() != TripStatus.PENDING)
            throw new BusinessException(
                "Cannot start trip. Status is: " + trip.getStatus()
                + ". Only PENDING trips can start.");

        trip.setStatus(TripStatus.IN_PROGRESS);
        trip.setStartedAt(LocalDateTime.now());

        // Vehicle becomes ON_TRIP
        trip.getVehicle().setStatus(VehicleStatus.ON_TRIP);
        vehicleRepository.save(trip.getVehicle());

        return toDto(tripRepository.save(trip));
    }

    // -- COMPLETE TRIP (IN_PROGRESS -> COMPLETED) ------------------------------
    @Transactional
    public TripResponseDto completeTrip(Long id) {
        Trip trip = findOrThrow(id);

        if (trip.getStatus() != TripStatus.IN_PROGRESS)
            throw new BusinessException(
                "Cannot complete trip. Status is: " + trip.getStatus()
                + ". Only IN_PROGRESS trips can be completed.");

        trip.setStatus(TripStatus.COMPLETED);
        trip.setCompletedAt(LocalDateTime.now());

        // Release vehicle back to AVAILABLE
        trip.getVehicle().setStatus(VehicleStatus.AVAILABLE);
        vehicleRepository.save(trip.getVehicle());

        return toDto(tripRepository.save(trip));
    }

    // -- CANCEL TRIP ------------------------------------------------------------
    @Transactional
    public TripResponseDto cancelTrip(Long id) {
        Trip trip = findOrThrow(id);
```

```java

        if (trip.getStatus() == TripStatus.COMPLETED
                || trip.getStatus() == TripStatus.CANCELLED)
            throw new BusinessException(
                "Cannot cancel a " + trip.getStatus() + " trip"
                );

        // If was IN_PROGRESS, release the vehicle
        if (trip.getStatus() == TripStatus.IN_PROGRESS) {
            trip.getVehicle().setStatus(VehicleStatus.AVAILABLE
                );
            vehicleRepository.save(trip.getVehicle());
        }

        trip.setStatus(TripStatus.CANCELLED);
        return toDto(tripRepository.save(trip));
    }

    // -- GET ALL
        ----------------------------------------------------------------

    public List<TripResponseDto> getAllTrips() {
        return tripRepository.findAll()
            .stream().map(this::toDto).collect(Collectors.
                toList());
    }

    // -- GET ONE
        ----------------------------------------------------------------

    public TripResponseDto getTripById(Long id) {
        return toDto(findOrThrow(id));
    }

    // -- MY TRIPS (DRIVER)
        ------------------------------------------------------
    public List<TripResponseDto> getMyTrips() {
        String username = SecurityContextHolder.getContext()
            .getAuthentication().getName();
        User user = userRepository.findByUsername(username).
            orElseThrow();
        Driver driver = driverRepository.findByUserId(user.
            getId())
            .orElseThrow(() -> new ResourceNotFoundException(
                "Driver profile not found for user: " +
                    username));

        return tripRepository.findByDriverId(driver.getId())
            .stream().map(this::toDto).collect(Collectors.
                toList());
    }
```

```java

    // -- PRIVATE ----------------------------------------------------------------

    private Trip findOrThrow(Long id) {
        return tripRepository.findById(id)
            .orElseThrow(() ->
                new ResourceNotFoundException("Trip not found: " + id));
    }

    private TripResponseDto toDto(Trip t) {
        return TripResponseDto.builder()
            .id(t.getId())
            .origin(t.getOrigin())
            .destination(t.getDestination())
            .status(t.getStatus())
            .notes(t.getNotes())
            .driverId(t.getDriver() != null ? t.getDriver().getId() : null)
            .driverName(t.getDriver() != null
                ? t.getDriver().getFirstName() + " "
                  + t.getDriver().getLastName() : null)
            .vehicleId(t.getVehicle() != null
                ? t.getVehicle().getId() : null)
            .vehiclePlate(t.getVehicle() != null
                ? t.getVehicle().getPlateNumber() : null)
            .vehicleType(t.getVehicle() != null
                ? t.getVehicle().getType().name() : null)
            .createdAt(t.getCreatedAt())
            .startedAt(t.getStartedAt())
            .completedAt(t.getCompletedAt())
            .build();
    }
}
```

Listing 8.3: TripService.java

```java
package com.fleetsync.controller;

import com.fleetsync.dto.TripRequestDto;
import com.fleetsync.dto.TripResponseDto;
import com.fleetsync.service.TripService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;
import java.util.List;

```

```java
@RestController
@RequestMapping("/api/trips")
@RequiredArgsConstructor
@CrossOrigin(origins = "*")
public class TripController {

    private final TripService tripService;

    @PostMapping
    @PreAuthorize("hasRole('MANAGER')")
    public ResponseEntity<TripResponseDto> create(
            @Valid @RequestBody TripRequestDto req) {
        return ResponseEntity.status(HttpStatus.CREATED)
                                .body(tripService.createTrip(req))
                                    ;
    }

    @GetMapping
    @PreAuthorize("hasAnyRole('MANAGER', 'ADMIN')")
    public ResponseEntity<List<TripResponseDto>> getAll() {
        return ResponseEntity.ok(tripService.getAllTrips());
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasAnyRole('MANAGER', 'ADMIN', 'DRIVER')")
    public ResponseEntity<TripResponseDto> getOne(@PathVariable
        Long id) {
        return ResponseEntity.ok(tripService.getTripById(id));
    }

    @GetMapping("/my-trips")
    @PreAuthorize("hasRole('DRIVER')")
    public ResponseEntity<List<TripResponseDto>> myTrips() {
        return ResponseEntity.ok(tripService.getMyTrips());
    }

    @PutMapping("/{id}/start")
    @PreAuthorize("hasAnyRole('MANAGER', 'DRIVER')")
    public ResponseEntity<TripResponseDto> start(@PathVariable
       Long id) {
        return ResponseEntity.ok(tripService.startTrip(id));
    }

    @PutMapping("/{id}/complete")
    @PreAuthorize("hasAnyRole('MANAGER', 'DRIVER')")
    public ResponseEntity<TripResponseDto> complete(
       @PathVariable Long id) {
        return ResponseEntity.ok(tripService.completeTrip(id));
    }

    @PutMapping("/{id}/cancel")
```

```java
    @PreAuthorize("hasRole('MANAGER')")
    public ResponseEntity<TripResponseDto> cancel(@PathVariable
        Long id) {
        return ResponseEntity.ok(tripService.cancelTrip(id));
    }
}
```

Listing 8.4: TripController.java

CHECKPOINT

Week 9 Checkpoint:

Create a trip – vehicle status visible in DB

Start trip – status becomes IN_PROGRESS, vehicle ON_TRIP

Complete trip – vehicle goes back to AVAILABLE

Delete ON_TRIP vehicle – get 400 error with message

Login as DRIVER, call /api/trips/my-trips – see only own trips

Login as DRIVER, try POST /api/trips – get 403 Forbidden

Chapter 9

Week 10: Polish – Swagger, Validation, API Response

9.1 Standard API Response Wrapper

```java
package com.fleetsync.dto;

import lombok.Builder;
import lombok.Data;
import java.time.LocalDateTime;

@Data
@Builder
public class ApiResponse<T> {

    private boolean success;
    private String message;
    private T data;
    private LocalDateTime timestamp;

    public static <T> ApiResponse<T> success(String msg, T data
        ) {
        return ApiResponse.<T>builder()
            .success(true).message(msg).data(data)
            .timestamp(LocalDateTime.now()).build();
    }

    public static <T> ApiResponse<T> error(String msg) {
        return ApiResponse.<T>builder()
            .success(false).message(msg).data(null)
            .timestamp(LocalDateTime.now()).build();
    }
}
```

Listing 9.1: ApiResponse.java

Update your controllers to wrap responses:

```
@PostMapping
@PreAuthorize("hasRole('MANAGER')")
public ResponseEntity<ApiResponse<VehicleResponseDto>> create(
        @Valid @RequestBody VehicleRequestDto req) {
    return ResponseEntity.status(HttpStatus.CREATED).body(
        ApiResponse.success("Vehicle created", vehicleService.
            createVehicle(req))
    );
}
```

Listing 9.2: Controller using ApiResponse wrapper

9.2 Swagger / OpenAPI Documentation

ROBOT – AI CAN WRITE THIS

Adding Swagger is just a dependency and two property lines.

Add to `pom.xml`:

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId
        >
    <version>2.3.0</version>
</dependency>
```

Listing 9.3: Swagger dependency

Add to `application.properties`:

```
springdoc.api-docs.path=/api-docs
springdoc.swagger-ui.path=/swagger-ui.html
```

Swagger UI will be live at http://localhost:8080/swagger-ui.html

Chapter 10

Week 11: Unit Testing

10.1 Why Testing Matters for Your Job Hunt

UAE and Pakistan senior devs ask: "Do you write tests?" If you say no, you look like a raw junior. If you say "yes, I wrote unit tests for my service layer using JUnit 5 and Mockito", you stand above 90% of other candidates.

CONCEPT NOTE

Unit Test vs Integration Test:

Unit Test: Tests one class in isolation. Dependencies are *mocked* (fake implementations). Runs in milliseconds. No database hit. Uses Mockito for mocks.

Integration Test: Tests multiple layers together (Controller + Service + Repository + real DB). Slower. Uses `@SpringBootTest`.

Writing even 5–10 solid unit tests shows you understand testing culture. That is enough for a junior portfolio.

10.2 VehicleService Unit Tests

HAND – WRITE THIS YOURSELF (CRITICAL)

Write these tests yourself. Testing IS writing code. It reinforces your understanding of the service logic and is directly asked about in interviews.

```java
package com.fleetsync.service;

import com.fleetsync.dto.VehicleRequestDto;
import com.fleetsync.dto.VehicleResponseDto;
import com.fleetsync.entity.Vehicle;
import com.fleetsync.enums.VehicleStatus;
import com.fleetsync.enums.VehicleType;
import com.fleetsync.exception.BusinessException;
import com.fleetsync.exception.DuplicateResourceException;
import com.fleetsync.exception.ResourceNotFoundException;
```

```java
import com.fleetsync.repository.VehicleRepository;
import org.junit.jupiter.api.*;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.*;
import org.mockito.junit.jupiter.MockitoExtension;
import java.util.Optional;
import static org.assertj.core.api.Assertions.*;
import static org.mockito.BDDMockito.*;

@ExtendWith(MockitoExtension.class)
class VehicleServiceTest {

    @Mock
    private VehicleRepository vehicleRepository;

    @InjectMocks
    private VehicleService vehicleService;

    private VehicleRequestDto validReq;
    private Vehicle savedVehicle;

    @BeforeEach
    void setUp() {
        validReq = new VehicleRequestDto();
        validReq.setPlateNumber("LEA-1234");
        validReq.setType(VehicleType.TRUCK);
        validReq.setModel("Toyota Hilux");
        validReq.setCapacity(1000);

        savedVehicle = Vehicle.builder()
            .id(1L).plateNumber("LEA-1234")
            .type(VehicleType.TRUCK).model("Toyota Hilux")
            .capacity(1000).status(VehicleStatus.AVAILABLE)
            .build();
    }

    @Test
    @DisplayName("Create: unique plate -> returns VehicleResponseDto")
    void create_uniquePlate_returnsDto() {
        given(vehicleRepository.existsByPlateNumber("LEA-1234"))
            .willReturn(false);
        given(vehicleRepository.save(any(Vehicle.class)))
            .willReturn(savedVehicle);

        VehicleResponseDto result = vehicleService.createVehicle(validReq);

        assertThat(result).isNotNull();
        assertThat(result.getPlateNumber()).isEqualTo("LEA-1234
```

```
            ");
        assertThat(result.getStatus()).isEqualTo(VehicleStatus.AVAILABLE);
        then(vehicleRepository).should(times(1)).save(any(Vehicle.class));
    }

    @Test
    @DisplayName("Create: duplicate plate -> throws DuplicateResourceException")
    void create_duplicatePlate_throws() {
        given(vehicleRepository.existsByPlateNumber("LEA-1234"))
            .willReturn(true);

        assertThatThrownBy(() -> vehicleService.createVehicle(validReq))
            .isInstanceOf(DuplicateResourceException.class)
            .hasMessageContaining("LEA-1234");

        then(vehicleRepository).should(never()).save(any());
    }

    @Test
    @DisplayName("GetById: not found -> throws ResourceNotFoundException")
    void getById_notFound_throws() {
        given(vehicleRepository.findById(999L)).willReturn(Optional.empty());

        assertThatThrownBy(() -> vehicleService.getVehicleById(999L))
            .isInstanceOf(ResourceNotFoundException.class);
    }

    @Test
    @DisplayName("Delete: ON_TRIP vehicle -> throws BusinessException")
    void delete_onTripVehicle_throws() {
        Vehicle onTrip = Vehicle.builder()
            .id(1L).status(VehicleStatus.ON_TRIP).build();
        given(vehicleRepository.findById(1L))
            .willReturn(Optional.of(onTrip));

        assertThatThrownBy(() -> vehicleService.deleteVehicle(1L))
            .isInstanceOf(BusinessException.class)
            .hasMessageContaining("ON_TRIP");
    }
}
```

Listing 10.1: VehicleServiceTest.java

Chapter 11

Week 12: Deployment and Portfolio

11.1 Dockerfile

ROBOT – AI CAN WRITE THIS

Docker configuration is infra boilerplate. Generate it and understand what it does.

```
# Stage 1: Build
FROM maven:3.9-eclipse-temurin-17-alpine AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Run (smaller image)
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/fleetsync-0.0.1-SNAPSHOT.jar app.
    jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Listing 11.1: Dockerfile in project root

11.2 Environment-Safe application.properties

```
spring.datasource.url=${DATABASE_URL:\
jdbc:mysql://localhost:3306/fleetsync_db}
spring.datasource.username=${DB_USERNAME:root}
spring.datasource.password=${DB_PASSWORD:}
spring.jpa.hibernate.ddl-auto=${DDL_AUTO:update}
spring.jpa.show-sql=false
jwt.secret=${JWT_SECRET:your-dev-secret-here}
jwt.expiration=${JWT_EXPIRATION:86400000}
server.port=${PORT:8080}
```

Listing 11.2: Production-ready application.properties

11.3 Deploy to Railway (Free)

1. Go to railway.app – sign up with GitHub
2. New Project → Deploy from GitHub Repo → select `fleetsync`
3. Add MySQL plugin: + New → Database → MySQL
4. Railway auto-sets `DATABASE_URL`, `MYSQLUSER`, `MYSQLPASSWORD`
5. Add `JWT_SECRET` in the Variables tab
6. Railway builds and deploys – you get a public URL

11.4 Resume Bullet Point

```
Built FleetSync, a Fleet Management REST API with Spring Boot
    3,
JWT authentication, RBAC for 3 user roles, and a trip lifecycle
state machine. Deployed on Railway with MySQL and documented
with Swagger/OpenAPI. github.com/YOUR_USERNAME/fleetsync
```

Listing 11.3: Resume line for FleetSync

INTERVIEW GOLD

"Walk me through your project" – use this script:

"FleetSync is a Fleet Management REST API built with Spring Boot 3. It manages vehicles, drivers, and trips for logistics companies. I implemented JWT-based stateless authentication with three roles – ADMIN, MANAGER, and DRIVER – using Spring Security and role-based access control via @PreAuthorize. The core business logic includes a trip state machine where vehicles automatically change from AVAILABLE to ON_TRIP when a trip starts and back to AVAILABLE on completion. I used Spring Data JPA with MySQL for persistence, Hibernate for ORM, and global exception handling via @ControllerAdvice. The app is deployed on Railway and documented with Swagger."

That answer covers: authentication, authorization, database, business logic, deployment, documentation. It passes junior screening.

Chapter 12

Interview Preparation – Top 20 Questions

12.1 Spring Boot

1. **Difference between @Component, @Service, @Repository?**
   All register a class as a Spring bean. Semantically different: @Service signals business logic, @Repository signals data access (also enables JPA exception translation), @Component is generic.

2. **What is Dependency Injection?**
   An object receives its dependencies from outside rather than creating them. Spring's application context creates and manages all beans. When a bean needs another, Spring injects it. Enables loose coupling and easy testing (inject mocks).

3. **@RestController vs @Controller?**
   @Controller is for MVC (returns view names). @RestController = @Controller + @ResponseBody, meaning return values go directly to HTTP response as JSON. For REST APIs always use @RestController.

4. **What is @Transactional?**
   Wraps the method in a DB transaction. If an exception is thrown, all DB changes in that method rollback. Use on service methods that do multiple writes that must all succeed or all fail.

5. **What is the N+1 query problem?**
   Loading 100 entities with EAGER relationships fires 1 query for the list + 100 queries for related data = 101 queries. Fix: use JOIN FETCH in a JPQL query to load everything in one SQL statement.

12.2 Security

6. **How does JWT work?**
   Login endpoint generates a signed token with user claims. Client sends it in every request via Authorization: Bearer header. Server verifies cryptographic signature and trusts the claims. No server-side session storage = stateless.

7. **Authentication vs Authorization?**
   Authentication = proving who you are (login). Authorization = what you are allowed to do (roles). JWT handles auth; @PreAuthorize handles authorization.

8. **Why BCrypt?**
   One-way hash – original password cannot be recovered. Adds random salt so identical passwords produce different hashes. Deliberately slow, making brute-force attacks infeasible.

9. **Why disable CSRF for REST APIs?**
   CSRF attacks exploit browser sessions. REST APIs are stateless (no sessions) and clients send JWT explicitly in headers – browsers don't include custom headers cross-site automatically. So CSRF protection is unnecessary.

10. **What is SessionCreationPolicy.STATELESS?**
    Tells Spring Security to never create or use HTTP sessions. Every request is independently authenticated via JWT.

12.3 JPA / Database

11. **What is ORM?**
    Object Relational Mapping. Maps Java classes to DB tables automatically. You define @Entity classes and Hibernate generates the SQL.

12. **LAZY vs EAGER?**
    LAZY loads related data only when accessed. EAGER loads immediately. LAZY is better for performance when you don't always need related data.

13. **Soft delete vs hard delete?**
    Hard delete removes the row permanently. Soft delete sets isActive = false. Soft delete preserves history and avoids foreign key issues. Industry standard for entities with relationships.

14. **What is JPQL?**
    Java Persistence Query Language. SQL-like but operates on entity objects, not table names. Example: `SELECT v FROM Vehicle v WHERE v.status = 'AVAILABLE'` – uses class name Vehicle, not table name vehicles.

12.4 General Java

15. **== vs .equals()?**
    == compares references (memory addresses). .equals() compares content. Always use .equals() for object comparison.

16. **What is Optional?**
    A container that may or may not hold a value. Avoids NullPointerException. Instead of returning null, return Optional.empty(). Caller is forced to handle the empty case.

17. **What is a Lambda?**
    A concise function representation. `.map(v -> v.getPlateNumber())` instead of an

anonymous class. Used with Java Streams for functional-style collection processing.

18. **What is the Java Stream API?**
Functional processing of collections. Key operations: map() (transform), filter() (select), collect() (terminate into a List), forEach() (iterate).

19. **What is @Builder?**
Lombok generates builder pattern. `Vehicle.builder().plateNumber("LEA").model("Toyota")` instead of multiple setters. Clean and readable.

20. **ArrayList vs LinkedList?**
ArrayList: dynamic array, O(1) random access, O(n) insert/delete in middle. LinkedList: doubly-linked nodes, O(n) access, O(1) insert/delete at known position. Use ArrayList for most Spring Boot work.

12.5 Pakistan and UAE Job Market Reality

Skill	Pakistan Market	UAE Market
Spring Boot	Mandatory	Mandatory
REST APIs	Mandatory	Mandatory
MySQL / PostgreSQL	Mandatory	Mandatory
Git / GitHub	Mandatory	Mandatory
JWT + Spring Security	Strong preference	Mandatory
Docker basics	Nice to have	Preferred
Unit Testing	Nice to have	Strong preference
Microservices	Bonus	Preferred
AWS basics	Bonus	Preferred

FleetSync covers: Spring Boot, REST APIs, MySQL, Git, JWT, Spring Security, Docker, Unit Testing. That is a strong junior portfolio for both markets.

Chapter 13

60-Day Execution Plan

Week	Focus	Deliverable
1–2	Java refresh + Spring Boot basics	Running app, first endpoint, GitHub repo
3–4	Database layer	All 4 entities, 4 repositories, tables created
5–6	CRUD APIs	All vehicle + driver endpoints in Postman
7	JWT Auth	Register + Login returning token
8	RBAC	Role-restricted endpoints working
9	Trip Module	Full lifecycle state machine working
10	Polish	Swagger UI live, ApiResponse wrapper
11	Testing	10+ unit tests written
12	Deployment	Live URL, clean README, demo ready

13.1 Job Search Parallel Track – Start Week 4

Do not wait for the project to be finished. Start applying in Week 4.

- **LinkedIn:** Update headline to "Java Developer | Spring Boot | Building Fleet Management System"
- **GitHub:** Commit daily. Activity graph shows work ethic.
- **Rozee.pk / Naukri:** Apply to junior Java / Spring Boot roles.
- **LinkedIn Easy Apply:** 5 applications per day. Volume matters.
- **Post weekly:** Share a code snippet or lesson from your project on LinkedIn. Pakistani dev community is supportive. Recruiters notice activity.

Chapter 14

AI Development Guide

14.1 The One Rule

WATCH OUT

Never paste code you do not understand.

Using AI to generate code you understand is a superpower. Using AI to generate code you do not understand is a trap that will destroy you in technical interviews.

Every time AI generates code: read it line by line. If you cannot explain what any single line does, ask AI to explain that line. Do not move on until you understand it.

14.2 Complete AI Policy Table

Component	Description	Policy
pom.xml	Maven dependencies	AI
application.properties	Configuration	AI
Dockerfile	Container config	AI
@Entity classes	Table definitions	Write Yourself
@Repository interfaces	JPA method signatures	AI (understand each method)
@Service classes	Business logic	Write Yourself
@RestController	Endpoint mapping	Write Yourself
DTOs	Request/Response shapes	Mixed
JwtUtils	Token generation/validation	Write Yourself
JwtAuthFilter	Security filter chain	AI (understand the flow)
SecurityConfig	Spring Security setup	AI (understand every @Bean)
Unit Tests	JUnit 5 + Mockito	Write Yourself
Exception classes	Custom exceptions	AI
GlobalExceptionHandler	Error handling	AI (understand the mappings)
README	Documentation	AI

14.3 Effective AI Prompt Patterns

14.3.1 Pattern 1: Explain Before Generate

```
Before writing the code, explain what a Spring Security Filter
Chain is and how JwtAuthFilter fits into it. Then write the
JwtAuthFilter class with a comment on EVERY line explaining
what it does.
```

14.3.2 Pattern 2: Debug Specific Errors

```
I'm getting this Spring Boot error:
[paste exact error from IntelliJ console]

My SecurityConfig looks like this:
[paste your code]

What is causing this and how do I fix it?
Explain why the fix works.
```

14.3.3 Pattern 3: Review Your Own Code

```
Review this Spring Boot service class I wrote.
Point out issues with:
1. Transaction handling
2. Business logic gaps
3. Exception handling
4. Naming conventions

[paste your code]

Do NOT rewrite it. Only point out issues and explain why.
```

Chapter 15

Phase 2 – AI Integration (After the Job)

15.1 The Architecture

Park this until you are employed. Then build it as Version 2. The pattern is a **microservices** architecture: Spring Boot calls a Python ML service via HTTP.

```
FleetSync (Spring Boot)  --HTTP-->  AI Service (Python/FastAPI)
        |                                        |
  MySQL DB                              ML Model (scikit-learn)
  (trips, drivers, vehicles)            (predicts performance scores
     )
```

Listing 15.1: Phase 2 microservice architecture

15.2 What the AI Adds

1. **Driver Performance Score:** trips completed, average duration, cancellation rate → score 0–100
2. **Trip Duration Prediction:** origin, destination, vehicle type, historical data → predicted hours
3. **Fleet Utilization Report:** which vehicles are underutilized, which drivers have been idle

15.3 Spring Boot Calling Python ML Service

```java
@Service
@RequiredArgsConstructor
public class AnalyticsService {

    private final TripRepository tripRepository;
    private final RestTemplate restTemplate;
```

```java
    private static final String AI_URL = "http://localhost:8000";

    public Double getDriverScore(Long driverId) {
        List<Trip> trips = tripRepository.findByDriverId(driverId);

        long completed = trips.stream()
            .filter(t -> t.getStatus() == TripStatus.COMPLETED).count();
        long cancelled = trips.stream()
            .filter(t -> t.getStatus() == TripStatus.CANCELLED).count();

        Map<String, Object> request = Map.of(
            "trips_completed", completed,
            "trips_cancelled", cancelled,
            "average_trip_duration_hours", 3.5,
            "on_time_rate", 0.85
        );

        Map response = restTemplate.postForObject(
            AI_URL + "/predict/performance", request, Map.class);

        return (Double) response.get("performance_score");
    }
}
```

Listing 15.2: AnalyticsService.java calling Python FastAPI

Appendix A

Appendix A: Complete pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.4.1</version>
    <relativePath/>
  </parent>

  <groupId>com.fleetsync</groupId>
  <artifactId>fleetsync</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
```

```
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-devtools</artifactId>
      <scope>runtime</scope>
      <optional>true</optional>
    </dependency>
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-api</artifactId>
      <version>0.12.3</version>
    </dependency>
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-impl</artifactId>
      <version>0.12.3</version>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-jackson</artifactId>
      <version>0.12.3</version>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.springdoc</groupId>
      <artifactId>springdoc-openapi-starter-webmvc-ui</
         artifactId>
      <version>2.3.0</version>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.security</groupId>
      <artifactId>spring-security-test</artifactId>
      <scope>test</scope>
    </dependency>
```

```
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
        <configuration>
          <excludes>
            <exclude>
              <groupId>org.projectlombok</groupId>
              <artifactId>lombok</artifactId>
            </exclude>
          </excludes>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Listing A.1: Full pom.xml

Appendix B

Appendix B: Postman Setup Guide

B.1 Collection Structure

Create a Postman collection "FleetSync API" with 4 folders: Auth, Vehicles, Drivers, Trips.

B.2 Auto-Save JWT Token

In your Login request Tests tab:

```
const r = pm.response.json();
if (r.token) {
    pm.collectionVariables.set("jwt_token", r.token);
    console.log("Token saved.");
}
```

Listing B.1: Postman test script

In all other requests Authorization header:

```
Authorization: Bearer {{jwt_token}}
```

Login once, all subsequent requests use the token automatically.

Appendix C

Appendix C: Common Errors and Fixes

Error	Fix
401 Unauthorized	Missing or invalid JWT in Authorization header
403 Forbidden	Token valid but user role lacks permission
BeanCreationException	Missing @Bean or circular dependency. Check @RequiredArgsConstructor
LazyInitializationException	Accessing lazy field outside transaction. Add @Transactional to service method
HttpMessageNotWritableException	Circular reference in JSON. Use DTOs instead of returning @Entity directly
DataIntegrityViolationException	Unique constraint violated. Handle with DuplicateResourceException
MySQL connection refused	MySQL not running. Start in XAMPP control panel
No qualifying bean	Missing @Service / @Repository / @Component on class

Now close this PDF and open IntelliJ.

The only thing left between you and the job is execution.
60 days. One project. Ship it.